Sentiment Analysis on IMDB Dataset [1]

- Objective: Classify movie reviews as positive or negative.
- Model: Fine-tune RoBERTa (Robustly Optimized BERT Pre-training)
- Output: Create a Python script that takes a movie review as input and predicts sentiment.

Introduction

Dataset description: IMDB dataset has 50K movie reviews with binary sentiment classification.

The goal of this project is to develop a machine learning model capable of accurately classifying the sentiment (either positive or negative) of these reviews based solely on the review text.

Install packages

```
! pip install transformers datasets evaluate accelerate scikit-learn torch
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.1)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: evaluate in /usr/local/lib/python3.12/dist-packages (0.4.6)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.11.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.36.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2025.3.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)
```

```
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.1.2)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.4.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11.1.6)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5.0)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (3.13.2)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (1.2.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.11.12)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
```

Load libraries

```
import torch
import evaluate
import numpy as np
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding
)

# Check for GPU availability
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
```

```
Using device: cpu
```

Methodology

Fine-tuen RoBERTa (Robustly Optimized BERT Pre-training), a transformer-based language model, on the IMDB dataset to classify movie reviews as positive or negative based on their content.

Measure its performance on unseen samples.

Specially, we use HuggingFace Transformers to fine-tune RoBERTa on the IMDB movie review dataset.

We use the Hugging Face datasets library to download the IMDB dataset. It contains 25,000 training examples and 25,000 test examples.

```
print("Loading IMDB dataset...")
dataset = load_dataset("imdb")
```

```
Loading IMDB dataset...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and rest
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

I want to test the code quickly without waiting hours, so I just use a small subset.

```
dataset["train"] = dataset["train"].select(range(200))
dataset["test"] = dataset["test"].select(range(50))
```

Data preprocessing - Tokenization

Convert the text into numbers (Input IDs) and Attention Masks. We also truncate reviews to 512 tokens, as that is RoBERTa's maximum input length.

```
model_checkpoint = "roberta-base"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

def preprocess_function(examples):
    # Truncation=True ensures inputs don't exceed the model's limit
    return tokenizer(examples["text"], truncation=True, max_length=512)

print("Tokenizing dataset (this may take a moment)...")
tokenized_datasets = dataset.map(preprocess_function, batched=True)
```

```
Tokenizing dataset (this may take a moment)...
Map: 100%                                                  50/50 [00:01<00:00, 30.65 examples/s]
```

Formatting the Data

- Rename columns: The dataset has a column named label, but the RoBERTa model expects an argument named labels.
- Remove text: The raw text is no longer needed after tokenization.
- Set Format: We must convert the lists to PyTorch tensors.

```
# Rename 'label' to 'labels'
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")

# Remove columns the model doesn't need
tokenized_datasets = tokenized_datasets.remove_columns(["text"])

# Set format to PyTorch tensors
tokenized_datasets.set_format("torch")

print("Data formatting complete.")
```

```
Data formatting complete.
```

Define metrics to measure accuracy during training and the loss.

```
accuracy_metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    # The model outputs logits; we take the argmax to get the predicted class (0 or 1)
    predictions = np.argmax(logits, axis=-1)
    return accuracy_metric.compute(predictions=predictions, references=labels)
```

Initialize a pre-trained RoBERTa model and trainer: load the pre-trained RoBERTa model and add a classification head with 2 labels then configure the training parameters.

```
# Define label mappings
id2label = {0: "NEGATIVE", 1: "POSITIVE"}
label2id = {"NEGATIVE": 0, "POSITIVE": 1}
```

```
# Load the model
model = AutoModelForSequenceClassification.from_pretrained(
    model_checkpoint,
    num_labels=2,
    id2label=id2label,
    label2id=label2id
)
```

```
Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized: ['classifier.dense.bias', 'c
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```
# Define Training Arguments
training_args = TrainingArguments(
    output_dir="./roberta_imdb_results",
    learning_rate=2e-5,              # Low learning rate for fine-tuning
    per_device_train_batch_size=4,  # Reduce to 8 or 4 if you get memory errors
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=2,   # Accumulate gradients to simulate batch_size=16
    fp16=True,                       # Use Mixed Precision (drastically reduces memory)
    gradient_checkpointing=True,     # Saves memory at the cost of slightly slower training
    num_train_epochs=2,              # 2 epochs is usually sufficient for IMDB
    weight_decay=0.01,
    eval_strategy="epoch",           # Evaluate at the end of every epoch
    save_strategy="epoch",           # Save model at the end of every epoch
    load_best_model_at_end=True,     # Load the best model when finished
    report_to="none"                 # DISABLES Weights & Biases logging
)
```

```
# Initialize the Data Collator (handles dynamic padding)
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

```
# Initialize the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
```

```
/tmp/ipython-input-3492237292.py:2: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for `Trainer.__init__`. Use `processing_class` instea
  trainer = Trainer(
```

Run the training

```
# Start Training
print("Starting Training...")
trainer.train()
```

```
Starting Training...
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device
  warnings.warn(warn_msg)
```

[50/50 56:58, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy
1	No log	0.002452	1.000000
2	No log	0.000836	1.000000

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device
  warnings.warn(warn_msg)
TrainOutput(global_step=50, training_loss=0.10990446090698242, metrics={'train_runtime': 3515.9303, 'train_samples_per_second': 0.114, 'train_steps_per_second':
0.014, 'total_flos': 83811148907760.0, 'train_loss': 0.10990446090698242, 'epoch': 2.0})
```

```
# Save the final model
trainer.save_model("./final_roberta_model")
print("Training complete and model saved.")
```

```
Training complete and model saved.
```

Results

Perform a manual test on a fake review to verify the model works.

```
# Test
print("\n--- Testing Model on New Data ---")
sample_text = "This movie was absolutely fantastic! The acting was great and the plot was moving."
```

```

--- Testing Model on New Data ---
```

```
# Tokenize the sample text
inputs = tokenizer(sample_text, return_tensors="pt")
```

```
# Move inputs to the same device as the model (GPU/CPU)
inputs = {k: v.to(model.device) for k, v in inputs.items()}
```

```
# Get predictions
with torch.no_grad():
    logits = model(**inputs).logits
```

```
# Convert logits to class ID
predicted_class_id = logits.argmax().item()
predicted_label = model.config.id2label[predicted_class_id]
```

```
print(f"Review: {sample_text}")
print(f"Predicted Sentiment: {predicted_label}")
```

```
Review: This movie was absolutely fantastic! The acting was great and the plot was moving.
Predicted Sentiment: NEGATIVE
```

Visualize Confusion Matrix

```
# Get predictions on the test set
predictions = trainer.predict(tokenized_datasets["test"])
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device
  warnings.warn(warn_msg)
```

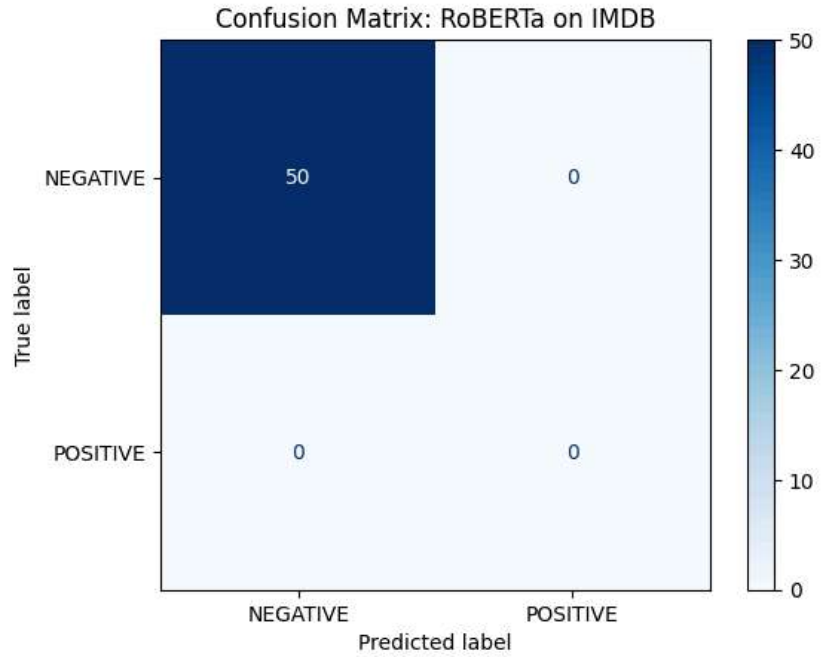
```
# Convert logits to class IDs (0 or 1)
preds = np.argmax(predictions.predictions, axis=-1)
true_labels = predictions.label_ids
```

```
# Compute and plot the confusion matrix

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(true_labels, preds, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["NEGATIVE", "POSITIVE"])
```

```
# Show the plot
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix: RoBERTa on IMDB")
plt.show()
```




```
# Print the confusion matrix
cm
```

```
array([[50,  0],
       [ 0,  0]])
```

Top-Left (50) is true negatives. The model correctly identified 50 reviews as "Negative."

Top-Right (0) is false positives. The model incorrectly predicted 0 reviews as "Positive."

Bottom-Left (0) is false negatives. The model incorrectly predicted 0 reviews as "Negative."

Bottom-Right (0) is true positives. The model correctly identified 0 reviews as "Positive."

The test set contained only negative reviews. The model tested on 50 samples, and all 50 of them were actually negative because we want to run the model faster, so I unintentionally choose an unbalanced test set.

Accuracy is 1. There is overfitting in the model.

Because there were no positive reviews in the test set, you don't know if the model can actually recognize a positive review, or if it's just predicting "negative" for everything.

Future improvements/scope.

We should have shuffled the dataset before selecting a subset.

We can do hyperparameter tuning by changing the learning rate and batch size.

We can use IMDB dataset contains raw HTML tags for better preprocessing the text.

References

[1] Lakshmi 25 N. Pathi. "IMDB Dataset of 50K Movie Reviews." Kaggle, www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews.